WM / (TM / VecSz) = warp 内横向 thread 分块数量, 不理解，请解释

下面我把 **WM / (TM / VecSz)** 这个表达式背后的含义讲得非常清楚，结合**warp 内线程如何分块负责一个 tile 的行方向(m 方向)存储任务**，让你完全理解它为什么是 “warp 内横向 thread 分块数量”。

🧩 一、先写出式子 & 定义

代码中：

```cpp
constexpr int VecSz = sizeof(VecT) / sizeof(T);
// 例: VecT = float4 → VecSz = 4

constexpr int TM_vec_num = TM / VecSz;   // 一个线程负责 TM 行，拆成 (TM / VecSz) 个 vector
constexpr int WM_per_loop = WM / TM_vec_num;
```

我们关注的式子就是：

```nginx
WM / (TM / VecSz)
```

也就是：

```ini
WM_per_loop = WM / TM_vec_num
```

🧩 二、每个参数的含义（非常关键）

参数	含义

🧩 二、每个参数的含义（非常关键）

参数	含义
WM	warp 负责的 tile 的行高（行方向 m）
TM	单个线程负责存的行数
VecSz	一个向量的元素数量（比如 float4 → 4）
TM / VecSz	一个线程内部分成多少个 vector chunk（沿 m 方向）
WM / (TM / VecSz)	warp 内 thread 横向划分出的子块数量

🧩 三、为什么是 “warp 内横向分块数量”

📌 warp 一共要存 WM 行

例如：

```ini
WM = 64 行
```

但一个线程并不是一次性处理 TM 行，而是分成：

```cpp
TM / VecSz 个 vector chunk
```

如果：

```ini
TM = 8, VecSz = 4 → TM_vec_num = 2
```

也就是说：

👉 每个线程在 m 方向被拆成 2 个 chunk（每 chunk ↓ ecSz = 4 行）。

也就是说：

👉 每个线程在 m 方向被拆成 2 个 chunk（每 chunk 是 VecSz = 4 行）。

那么：

```nginx
warp 中的每一次 tm_loop，都要覆盖整个 WM 行。
```

🧩 四、那么 warp 要如何覆盖整个 WM 行？

warp 中每条线程都有不同的：

```nginx
tm_vec_id
```

每一次 tm_loop 时，每个线程应该负责一个 m 子区间，因此 m 方向的 warp 划分数量是：

```nginx
m 子区块数量 = WM / (每次迭代每个线程覆盖的行数)
```

而每次迭代内：

```cpp
每线程覆盖行数 = VecSz (vector 幅度)
```

且 tm_loop 有 TM / VecSz 个，总共覆盖 TM 行，但一次循环推进 VecSz 行。

因此：

```cpp
warp 内横向 thread 分块数量 = WM / (TM_vec_num)
```

也就是：

🧩 四、那么 warp 要如何覆盖整个 WM 行?

warp 中每条线程都有不同的:

```nginx
tm_vec_id
```

每一次 tm_loop 时，每个线程应该负责一个 m 子区间，因此 m 方向的 warp 划分数量是:

```nginx
m 子区块数量 = WM / (每次迭代每个线程覆盖的行数)
```

而每次迭代内:

```cpp
每线程覆盖行数 = VecSz (vector 幅度)
```

且 tm_loop 有 TM / VecSz 个，总共覆盖 TM 行，但一次循环推进 VecSz 行。

因此:

```cpp
warp 内横向 thread 分块数量 = WM / (TM_vec_num)
```

也就是:

```nginx
WM / (TM / VecSz)
```

🧩 五、具体数字例子（最容易理解）

假设:

- WM = 64 （warp tile 高度）

也就是：

```nginx
WM / (TM / VecSz)
```

🧩 五、具体数字例子（最容易理解）

假设：

- WM = 64 （warp tile 高度）
- TM = 8 （每线程计算 8 行）
- VecSz = 4（float4）

1️⃣ 每个线程内部 vector chunk 数：

```ini
TM_vec_num = 8 / 4 = 2
```

即：一个线程在 m 方向要分 2 次存储，每次存 4 行。

2️⃣ warp 内分成多少“行方向子块”？

```ini
WM_per_loop = WM / TM_vec_num = 64 / 2 = 32
```

解释：

-整个 warp 每一个 tm_loop 迭代覆盖 **32 行**
-因为需要 2 次 tm_loop 才覆盖到全部 64 行

3️⃣ 那 warp 内 thread 如何分配？

3️⃣ 那 warp 内 thread 如何分配？

每个 warp 有多个线程，每个线程有一个 **tm_vec_id**：

tm_vec_id	负责的行区间（每次 Vid）
0	第 0~3 行（VecSz=4）
1	第 32~35 行

因此：

```ini
tm_vec_id == 0 处理  0~31 行区间（每次 tid 移动 4 行）
tm_vec_id == 1 处理 32~63 行区间
```

正好两半。

🧩 六、因此为什么它是 “warp 内横向 thread 分块数量”

因为：

```cpp
warp 需要将 WM 行分成（TM / VecSz）个区域
每个 thread 负责其中一个子区域（由 tm_vec_id 控制）
```

所以：

```nginx
WM / (TM / VecSz)
= 每个这样的子区域要覆盖的行数
= warp 在行方向的划分数量
```

也就是说：

📜 **warp 内线程在行方向被划分成 TM_vec_num 组**
📜 **每组负责 WM / TM_vec_num 行**

🧩 六、因此为什么它是 “warp 内横向 thread 分块数量”

因为：

```cpp
warp 需要将 WM 行分成（TM / VecSz）个区域
每个 thread 负责其中一个子区域（由 tm_vec_id 控制）
```

所以：

```nginx
WM / (TM / VecSz)
= 每个这样的子区域要覆盖的行数
= warp 在行方向的划分数量
```

也就是说：

- 🏷️ **warp 内线程在行方向被划分成 TM_vec_num 组**
- 🏷️ **每组负责 WM / TM_vec_num 行**
- 🏷️ **这个数量就是 WM / (TM / VecSz)**

🧩 七、一句话总结（非常精炼）

WM / (TM / VecSz) 表示：warp 在 M 方向上根据每线程的“vector 行数”划分出的子块高度，每个 tm_vec_id 对应一个这样的子块。它就是 warp 内线程在行方向负责的 tile 分块数量。

如果你愿意，我可以进一步提供：

- ✅ 全流程 ASCII 图示
- ✅ warp 32 个线程的完整责任行映射图
- ✅ 用真实 CUDA 模拟代码打印 m,n 访问顺序
- ✅ 画图解释 register→global 的存储分布

只需要告诉我：“继续画图” 或 “继续举例”。

//—-—-—-

✅ 1. Warp Tile 被按 thread-group（tm_vec_id）切成条状

假设：

```ini
WM = 64
TM = 8
VecSz = 4
TM_vec_num = TM / VecSz = 2
WM_per_loop = WM / TM_vec_num = 64 / 2 = 32
```

warp tile（64 行）被 thread 组分成两部分：

```sql
Warp Tile (64 rows)
┌──────────────────────────┐
│ tm_vec_id = 0 → rows 0-31  │  (warp 中部分线程)
├──────────────────────────┤
│ tm_vec_id = 1 → rows 32-63 │  (warp 中另一部分线程)
└──────────────────────────┘
```

效果：

warp 内线程按 tm_vec_id 划分成 2 组，每组负责不同的行块。

✅ 2. 每个 Warp Tile 再按 tn_vec_id 划纵向划分（列方向）

假设：

```ini
WN = 32
TN = 8
VecSz=4 → TN_vec_num=2
WN_per_loop = 32 / 2 = 16
```

```csharp
Warp Tile (32 columns)
```

✅ 2. 每个 Warp Tile 再按 tn_vec_id 划纵向划分（列方向）

假设：

```ini
WN = 32
TN = 8
VecSz=4 → TN_vec_num=2
WN_per_loop = 32 / 2 = 16
```

```csharp
Warp Tile (32 columns)
┌──────────────┬──────────────┐
│ tn_vec_id=0  │ tn_vec_id=1  │
│ cols 0–15    │ cols 16–31   │
└──────────────┴──────────────┘
```

✅ 3. Warp Tile 最终划分成 Thread 承担的区域（组合 tm_vec_id + tn_vec_id）

```sql
Warp Tile (WM x WN)
64 x 32
┌──────────────────────────────────────────┐
│ tm_vec_id=0, tn_vec_id=0  → rows 0–31, cols 0–15  │
│ tm_vec_id=0, tn_vec_id=1  → rows 0–31, cols 16–31 │
│ tm_vec_id=1, tn_vec_id=0  → rows 32–63, cols 0–15 │
│ tm_vec_id=1, tn_vec_id=1  → rows 32–63, cols 16–31│
└──────────────────────────────────────────┘
```

每个 thread 由 (tm_vec_id , tn_vec_id) 决定自己储存哪一块。

✅ 4. 单个 Thread Tile（TM x TN）示意

✅ 4. 单个 Thread Tile（TM x TN）示意

例如：

```ini
TM = 8
TN = 8
```

```sql
Thread Tile (8 x 8)
┌──────────────────────┐
│ row 0 ... row 7      │  <-- computed in registers tCrC
│ col 0 ... col 7      │
└──────────────────────┘
```

✅ 5. Thread Tile 再被 Vector Tile 划分（VecSz = 4）

沿 m 方向（TM）拆成 TM_vec_num=2 个 vector：

```sql
Thread Tile 8 rows
┌──────────────┬──────────────┐
│ 4 rows (vec0)│ 4 rows (vec1) │
│ _m = 0..3    │ _m = 4..7     │
└──────────────┴──────────────┘
```

沿 n 方向（TN）也拆成 TN_vec_num=2：

```mathematica
Thread Tile 8 columns
┌──────────────┬──────────────┐
│ 4 cols (v0)  │ 4 cols (v1)  │
│ _n=0..3      │ _n=4..7      │
└──────────────┴──────────────┘
```

✅ 6. Vector Tile 访问示意（最终写入 global）

每一次写入：

```scss
get_offset(VecT, tCtCgC, m, n)  ←  get_offset(VecT, tCrC, _m, _n)
```

ASCII 示意：

```csharp
REGISTER TILE (tCrC, TM x TN)
┌──────────────┬──────────────┐
│ (_m=0..3,_n=0..3) │ (_m=0..3,_n=4..7) │  tm_loop=0
├──────────────┼──────────────┤
│ (_m=4..7,_n=0..3) │ (_m=4..7,_n=4..7) │  tm_loop=1
└──────────────┴──────────────┘
```

写到 Global：

```java
GLOBAL TILE (Warp Tile)
m = tm_vec_id*(WM_per_loop) + tm_loop*VecSz + vid
n = tn_vec_id*(WN_per_loop) + tn_loop*VecSz
```

✅ 7. 全流程总示意（最终版）

```sql
GLOBAL C MATRIX
┌──────────────────────────────────────────────────────────────┐
│ BLOCK TILE                                                   │
│ ┌──────────────────────────────────────────────────────────┐ │
│ │ WARP TILE (WM x WN)                                      │ │
│ │  tm_vec_id=0 → rows 0-31                                 │ │
│ │  tm_vec_id=1 → rows 32-63                                │ │
│ │                        tn_vec_id=0 (col  -15)            │ │
│ │                        tn_vec_id=1 (cols 16-31)          │ │
```

✅ 7. 全流程总示意（最终版）

```sql
GLOBAL C MATRIX
┌──────────────────────────────────────────────────────────────┐
│ BLOCK TILE                                                   │
│ ┌──────────────────────────────────────────────────────────┐ │
│ │ WARP TILE (WM x WN)                                      │ │
│ │  tm_vec_id=0 → rows 0-31                                 │ │
│ │  tm_vec_id=1 → rows 32-63                                │ │
│ │                        tn_vec_id=0 (cols 0-15)           │ │
│ │                        tn_vec_id=1 (cols 16-31)          │ │
│ │  ┌──────────────────────────────────────────────┐        │ │
│ │  │ THREAD TILE (TM x TN = 8x8)                  │        │ │
│ │  │  divided into vector tiles (4x4)             │        │ │
│ │  │   (_m,_n) loops: tm_loop=0..1, tn_loop=0..1  │        │ │
│ │  │                                              │        │ │
│ │  │   REGISTER TILE tCrC ---> GLOBAL TILE tCgC   │        │ │
│ │  └──────────────────────────────────────────────┘        │ │
│ └──────────────────────────────────────────────────────────┘ │
└──────────────────────────────────────────────────────────────┘
```

如果你希望，我可以继续提供：

- ✅ 每个线程（tid=0..31）的具体 m,n 写入坐标表
- ✅ 真实参数下（WM=64,WN=32,TM=8,TN=8）完整访问顺序打印
- ✅ 线程对齐 & 向量存储的合并写访存解释
- ✅ 完整 C++/CUDA 小代码模拟版，打印每次访问

只需要告诉我："继续打印 thread 访问表" 或 "继续画更细粒度的图"。

//————————————

然后：

```cpp
每个线程负责其中：每次写 1 行（VecSz=4 的 vector）
```

✅ 最终映射：warp 32 个线程 → 64 行的责任行完整图

下面是完整 ASCII 图：

（左边是行号，右边是负责该行的 thread lane）

```mathematica
          WARP 32 THREAD → 64 ROWS RESPONSIBILITY MAP
═══════════════════════════════════════════════════════
  Row  0  ← lane  0
  Row  1  ← lane  1
  Row  2  ← lane  2
  Row  3  ← lane  3
  Row  4  ← lane  4
  Row  5  ← lane  5
  Row  6  ← lane  6
  Row  7  ← lane  7
  Row  8  ← lane  8
  Row  9  ← lane  9
  Row 10  ← lane 10
  Row 11  ← lane 11
  Row 12  ← lane 12
  Row 13  ← lane 13
  Row 14  ← lane 14
  Row 15  ← lane 15
───────────────────────────────────────────────────────
  Row 16  ← lane 16
  Row 17  ← lane 17
  Row 18  ← lane 18
  Row 19  ← lane 19
  Row 20  ← lane 20
  Row 21  ← lane 21
  Row 22  ← lane 22
```

```
Row 16  ← lane 16
Row 17  ← lane 17
Row 18  ← lane 18
Row 19  ← lane 19
Row 20  ← lane 20
Row 21  ← lane 21
Row 22  ← lane 22
Row 23  ← lane 23
Row 24  ← lane 24
Row 25  ← lane 25
Row 26  ← lane 26
Row 27  ← lane 27
Row 28  ← lane 28
Row 29  ← lane 29
Row 30  ← lane 30
Row 31  ← lane 31
Row 32  ← lane  0
Row 33  ← lane  1
Row 34  ← lane  2
Row 35  ← lane  3
Row 36  ← lane  4
Row 37  ← lane  5
Row 38  ← lane  6
Row 39  ← lane  7
Row 40  ← lane  8
Row 41  ← lane  9
Row 42  ← lane 10
Row 43  ← lane 11
Row 44  ← lane 12
Row 45  ← lane 13
Row 46  ← lane 14
Row 47  ← lane 15
Row 48  ← lane 16
Row 49  ← lane 17
Row 50  ← lane 18
Row 51  ← lane 19
Row 52  ← lane 20
```

```
Row 48  ← lane 16
Row 49  ← lane 17
Row 50  ← lane 18
Row 51  ← lane 19
Row 52  ← lane 20
Row 53  ← lane 21
Row 54  ← lane 22
Row 55  ← lane 23
Row 56  ← lane 24
Row 57  ← lane 25
Row 58  ← lane 26
Row 59  ← lane 27
Row 60  ← lane 28
Row 61  ← lane 29
Row 62  ← lane 30
Row 63  ← lane 31
```

✅ 解释：为什么每个 lane 负责 2 行？

因为：

```makefile
WM = 64 行
warp 只有 32 线程
→ 每线程必须覆盖 2 行
```

分配方式，由 TM_vec_id 决定：

```css
lane in [0..31]:
    if (lane < 32):
        tm_vec_id = lane / 32 = 0    → rows 0~31
    if (lane < 32):
        tm_vec_id = 0 or 1 取决于 block 的分
```

✅ 解释：为什么每个 lane 负责 2 行？

因为：

```makefile
WM = 64 行
warp 只有 32 线程
→ 每线程必须覆盖 2 行
```

分配方式，由 TM_vec_id 决定：

```css
lane in [0..31]:
    if (lane < 32):
        tm_vec_id = lane / 32 = 0    → rows 0~31
    if (lane < 32):
        tm_vec_id = 0 or 1 取决于 block 的分块
```

实际更准确：

- **tm_vec_id = threadIdx.y**（如果你 warp 排布是 32x1）
- 或你自定义 grid 也可能：`tm_vec_id = lane / (warpSize/2)`

但因为你例子：

```ini
TM = 8
VecSz = 4 → TM_vec_num = 2
WM = 64 → WM_per_loop = 32
```

所以 mapping 就是：

```sql
每个 lane 负责一条 row，分别在上 32 行和下 32 行各一条
```

🔥 最终总结（口语版）

🔥 最终总结（口语版）

整个 warp：32 个线程
需要覆盖 64 行 tile

所以：

- **前 32 行：lane 0~31**
- **后 32 行：lane 0~31 再负责一次**

这就是为什么你看到：

```scss
Row X → lane (X % 32)
```

如果你需要，我可以继续画：

✅ **Warp 写入 global memory 的完整二维映射**

✅ **(thread, tm_loop, tn_loop, vecSz) → 真实坐标 (m,n) 的完整矩阵图**

✅ **含 vector store 的合并写访存示意**

✅ **每个 lane 写入的所有 C[m][n] 的完整列表**

只要告诉我："继续画二维图" 或 "给我每个 lane 的完整写入坐标表"。

//—-—-—-

- **BN** = 128：Block处理B矩阵的列数
- **WN** = 32：Warp处理B矩阵的列数
- **TN** = 8：Thread处理B矩阵的列数
- **vec_size** = 4（假设使用float4/uint4）
- **TN_vec_num** = TN / vec_size = 8 / 4 = 2
- **WN_per_loop** = WN / TN_vec_num = 32 / 2 = 16

2. WN_per_loop的物理意义

`WN_per_loop` 表示：**一个Warp的WN列数据需要分成多少次循环来加载到寄存器中，每次循环处理的数据跨度**。

具体来说：
- 一个Warp负责处理 `WN=32` 列数据
- 一个Thread负责处理 `TN=8` 列数据
- 但由于使用向量化（vec_size=4），实际每次加载 `vec_size=4` 个元素
- 所以每个Thread需要 `TN_vec_num = TN/vec_size = 2` 次循环来完成自己TN列数据的加载
- 每次循环中，不同Thread加载**不同列**的数据

3. 图示说明

3.1 全局视图（Block级别）
```
BN = 128列

┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│                                                                                                                                │
│  WN=32 列      WN=32 列      WN=32 列      WN=32 列                                                                            │
│  ┌──────────────────────┐  ┌──────────────────────┐  ┌──────────────────────┐  ┌──────────────────────┐                    │
│  │ Warp 0    │  │ Warp 1    │  │ Warp 2    │  │ Warp 3    │                    │
│  │ wn_id=0   │  │ wn_id=1   │  │ wn_id=2   │  │ wn_id=3   │                    │
│  └──────────────────────┘  └──────────────────────┘  └──────────────────────┘  └──────────────────────┘                    │
│                                                                                                                                │
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
```

3.2 Warp级别（WN=32列）
```
一个Warp的WN=32列如何分配给32个Thread：
  WN_per_loop = 16列/循环
```

```
  循环0 (tn_loop=0):                循环1 (tn_loop=1):
  ┌──────────────────────────────────────────┐
┌──────────────────────────────────────────┐
  │   Thread 0-15       |              |   Thread 0-15       |
  │  加载列0-15         |              |  加载列16-31        |
  │  每个Thread加载1列  |              |  每个Thread加载1列  |
  └──────────────────────────────────────────┘
└──────────────────────────────────────────┘
```

3.3 Thread内部的向量化加载

```
Thread 0 的数据加载模式（tn_vec_id=0）:
WN_per_loop = 16，vec_size = 4

  循环0 (tn_loop=0):                循环1 (tn_loop=1):
  列: 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
     ↑           ↑
     Thread 0加载列0-3  (向量化，一次4列)

  循环0 (tn_loop=0):                循环1 (tn_loop=1):
  列: 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
     ↑           ↑
     Thread 0加载列16-19 (向量化，一次4列)
```

3.4 详细计算过程

```cpp
// tn_vec_id 决定了Thread在WN_per_loop中的位置
// tn_loop 决定是第几次循环

int n = tn_loop * WN_per_loop + tn_vec_id * vec_size;

当 tn_vec_id=0, tn_loop=0: n = 0*16 + 0*4 = 0    // 加载列0-3
当 tn_vec_id=0, tn_loop=1: n = 1*16 + 0*4 = 16   // 加载列16-19

当 tn_vec_id=1, tn_loop=0: n = 0*16 + 1*4 = 4    // 加载列4-7
当 tn_vec_id=1, tn_loop=1: n = 1*16 + 1*4 = 20   // 加载列20-23

当 tn_vec_id=2, tn_loop=0: n = 0*16 + 2*4 = 8    // 加载列8-11
当 tn_vec_id=2, tn_loop=1: n = 1*16 + 2*4 = 24   // 加载列24-27

当 tn_vec_id=3, tn_loop=0: n = 0*16 + 3*4 = 12   // 加载列12-15
当 tn_vec_id=3, tn_loop=1: n = 1*16 + 3*4 = 28   // 加载列28-31
```

```

### 3.5 完整映射图

```
WN = 32列，TN_vec_num = 2次循环，WN_per_loop = 16列/循环

Thread分布（根据tn_vec_id）：
tn_vec_id=0: Thread 0,4,8,12,16,20,24,28 → 加载列{0-3,16-19}
tn_vec_id=1: Thread 1,5,9,13,17,21,25,29 → 加载列{4-7,20-23}
tn_vec_id=2: Thread 2,6,10,14,18,22,26,30 → 加载列{8-11,24-27}
tn_vec_id=3: Thread 3,7,11,15,19,23,27,31 → 加载列{12-15,28-31}

循环0 (tn_loop=0) - 列0-15:
列: 0-3   4-7   8-11 12-15
   [t0] [t1] [t2] [t3]   ← tn_vec_id=0-3的Thread
   [t4] [t5] [t6] [t7]
   [t8] [t9] [t10] [t11]
   [t12] [t13] [t14] [t15]
   [t16] [t17] [t18] [t19]
   [t20] [t21] [t22] [t23]
   [t24] [t25] [t26] [t27]
   [t28] [t29] [t30] [t31]

循环1 (tn_loop=1) - 列16-31:
列: 16-19 20-23 24-27 28-31
   [t0] [t1] [t2] [t3]   ← tn_vec_id=0-3的Thread
   [t4] [t5] [t6] [t7]
   [t8] [t9] [t10] [t11]
   [t12] [t13] [t14] [t15]
   [t16] [t17] [t18] [t19]
   [t20] [t21] [t22] [t23]
   [t24] [t25] [t26] [t27]
   [t28] [t29] [t30] [t31]
```

## 4. 代码执行流程

```cpp
// 对于每个Thread，执行2次循环（TN_vec_num=2）
#pragma unroll
for (int tn_loop = 0; tn_loop < TN_vec_num; ++tn_loop) {
    // 计算要加载的列号
    // tn_loop=0: 加载前WN_per_loop列（0-15列）
    // tn_loop=1: 加载后WN_per_loop列（16-31列）
    int n = tn_loop * WN_per_loop + tn_vec_id * vec_size;
```
```

```
    // 计算寄存器中的偏移
    int _n = tn_loop * vec_size;

    // 向量化加载
    get_offset(vec_t, tBrB, kid, _n, TN)[0] =
        get_offset(vec_t, tBsB, kid, n, BN)[0];
}
```

5. 为什么需要WN_per_loop?

1. **内存访问合并**：通过`WN_per_loop`将32个Thread的访问组织成连续的向量化访问，提高内存带宽利用率。

2. **寄存器压力控制**：每个Thread只加载TN=8个元素到寄存器，但通过向量化分成2次完成，平衡了计算和访存。

3. **数据重用**：Warp内的Thread共享数据加载，减少重复加载。

4. **并行性**：32个Thread同时加载不同列的数据，充分利用GPU的并行性。

6. 总结

`WN_per_loop = 16` 的物理意义是：**在一个Warp处理WN=32列数据时，每次循环处理连续16列数据，这些16列数据由32个Thread协作加载，每个Thread通过向量化加载其中的vec_size=4列**。

这种设计实现了：
- **高效的内存访问**：合并访问模式
- **负载均衡**：所有Thread工作量相同
- **数据局部性**：连续的内存访问
- **向量化优势**：利用向量加载指令提高吞吐量

//—-—-—-

1. 基本定义

从代码中看：
```cpp
const int tn_vec_id = lane_id & (WN_per_loop/vec_size-1);
```

其中：
- `lane_id` = thread在warp内的索引（0-31）
- `WN_per_loop` = 16
- `vec_size` = 4
- `WN_per_loop/vec_size` = 16/4 = 4
- `WN_per_loop/vec_size-1` = 3 = 0b11

所以：
```cpp
tn_vec_id = lane_id & 3  // 取lane_id的低2位
```

2. 物理意义

`tn_vec_id` 表示：**线程在WN_per_loop范围内的列组中的位置**。

具体来说：
- WN_per_loop=16列数据被分成4个组（每组vec_size=4列）
- tn_vec_id=0,1,2,3 分别对应这4个组
- 每个组内的多个线程并行加载同一组数据

3. 举例说明

3.1 线程分配示例

假设有8个线程（简化示例），vec_size=4：
```
线程ID (lane_id):  0  1  2  3  4  5  6  7
tn_vec_id:         0  1  2  3  0  1  2  3
                   ↑  ↑  ↑  ↑  ↑  ↑  ↑  ↑
列组:              组0 组1 组2 组3 组0 组1 组2 组3
```

3.2 实际加载模式（完整32线程）

完整32个线程的`tn_vec_id`分配：
```
lane_id: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
tn_vec_id:0 1 2 3 0 1 2 3 0 1 2  3  0  1  2  3  0  1  2  3  0  1  2  3  0  1  2  3  0  1  2  3
```

3.3 加载的具体列数据

当 `tn_loop=0`（加载前16列）时：

```
WN_per_loop=16列分为4个组：
组0: 列0-3     (tn_vec_id=0的线程加载)
组1: 列4-7     (tn_vec_id=1的线程加载)
组2: 列8-11    (tn_vec_id=2的线程加载)
组3: 列12-15   (tn_vec_id=3的线程加载)

具体哪个线程加载：
列0-3:   线程0,4,8,12,16,20,24,28 (所有tn_vec_id=0的线程)
列4-7:   线程1,5,9,13,17,21,25,29 (所有tn_vec_id=1的线程)
列8-11:  线程2,6,10,14,18,22,26,30 (所有tn_vec_id=2的线程)
列12-15: 线程3,7,11,15,19,23,27,31 (所有tn_vec_id=3的线程)
```

3.4 计算过程示例

代码中的关键计算：
```cpp
int n = tn_loop * WN_per_loop + tn_vec_id * vec_size;
```

示例1：线程0 (lane_id=0, tn_vec_id=0)
```
tn_loop=0: n = 0*16 + 0*4 = 0    → 加载列0-3
tn_loop=1: n = 1*16 + 0*4 = 16   → 加载列16-19
```

示例2：线程1 (lane_id=1, tn_vec_id=1)
```
tn_loop=0: n = 0*16 + 1*4 = 4    → 加载列4-7
tn_loop=1: n = 1*16 + 1*4 = 20   → 加载列20-23
```

示例3：线程3 (lane_id=3, tn_vec_id=3)
```
tn_loop=0: n = 0*16 + 3*4 = 12   → 加载列12-15
tn_loop=1: n = 1*16 + 3*4 = 28   → 加载列28-31
```

4. 可视化图示

4.1 列分配图
```
WN = 32列（一个Warp的列范围）
分为2次循环加载（TN_vec_num=2）：

循环0 (tn_loop=0): 列0-15
┌──────────────────────────────────────────────────────────────┐
│ 组0      组1      组2      组3        │
│ (0-3)   (4-7)   (8-11)  (12-15)      │
│  tn=0   tn=1    tn=2    tn=3         │
│  8个线程 8个线程 8个线程 8个线程      │
└──────────────────────────────────────────────────────────────┘

循环1 (tn_loop=1): 列16-31
┌──────────────────────────────────────────────────────────────┐
│ 组0      组1      组2      组3        │
│ (16-19) (20-23) (24-27) (28-31)       │
│  tn=0   tn=1    tn=2    tn=3         │
│  8个线程 8个线程 8个线程 8个线程      │
└──────────────────────────────────────────────────────────────┘
```

4.2 线程-列映射详图
```
Thread Mapping for tn_vec_id:

tn_vec_id=0的线程 (lane_id的低2位为00):
线程: 0,4,8,12,16,20,24,28
加载列: 循环0:列0-3, 循环1:列16-19

tn_vec_id=1的线程 (lane_id的低2位为01):
线程: 1,5,9,13,17,21,25,29
加载列: 循环0:列4-7, 循环1:列20-23

tn_vec_id=2的线程 (lane_id的低2位为10):
线程: 2,6,10,14,18,22,26,30
加载列: 循环0:列8-11, 循环1:列24-27

tn_vec_id=3的线程 (lane_id的低2位为11):
线程: 3,7,11,15,19,23,27,31
加载列: 循环0:列12-15, 循环1:列28-31
```

5. 为什么这样设计?

5.1 内存访问模式优化
```
共享内存中的访问模式：
假设sB是行主序：[BK][BN] = [16][128]

当所有tn_vec_id=0的线程加载列0-3时：
线程0: sB[kid][0:4]  ← 连续访问
线程4: sB[kid][0:4]  ← 相同数据（广播）
...
这样设计可以实现：
1. 合并访问：同一组的线程访问连续内存
2. 数据复用：多个线程共享同一数据
```

5.2 与寄存器布局的对应
```
寄存器tBrB的布局：WK×TN = 8×8
每个线程有8个寄存器元素（TN=8），通过2次循环加载：

循环0 (tn_loop=0): 加载到寄存器位置0-3
循环1 (tn_loop=1): 加载到寄存器位置4-7

寄存器索引计算：_n = tn_loop * vec_size
所以：
tn_loop=0: _n=0 → 寄存器位置0-3
tn_loop=1: _n=4 → 寄存器位置4-7
```

5.3 SIMD效率最大化
```
vec_size=4 对应GPU的128位加载指令
每个线程一次加载4个float（一个向量）
32个线程 × 4元素/线程 = 128元素/周期
正好可以充分利用内存带宽
```

6. 物理意义的总结

`tn_vec_id` 的核心物理意义是：

1. **列组标识符**：标识线程属于WN_per_loop范围内的哪个列组（每组vec_size列）

2. **数据分配器**：决定线程负责加载哪些具体的列数据

3. **线程分类器**：将warp内的32个线程分为4组（因为WN_per_loop/vec_size=4）

4. **内存访问组织者**：确保同一组的线程访问连续的内存区域，实现合并访问

5. **计算并行化基础**：为后续的矩阵乘法计算准备数据布局，使每个线程有完整的TN列数据进行计算

7. 与tm_vec_id的对比

```
对于A矩阵（M维度）：
tm_vec_id = lane_id >> get_log2x<WN_per_loop/vec_size>();

对于B矩阵（N维度）：
tn_vec_id = lane_id & (WN_per_loop/vec_size-1);

这体现了不同的划分策略：
- M维度：通过右移位划分（按WN_per_loop分组）
- N维度：通过掩码划分（在WN_per_loop内分组）
```

这种不对称的划分是因为：
- A矩阵在共享内存中是转置的（BK×BM）
- B矩阵在共享内存中是正常的（BK×BN）
- 不同的内存布局需要不同的访问模式优化

//————————————

- **WM_per_loop** = WM / TM_vec_num = 64 / 2 = 32

2. 物理意义详解

`WM_per_loop = 32` 的物理意义是：
在一个Warp处理WM=64行数据时，每次循环处理连续32行数据，这些32行数据由32个Thread协作加载，每个Thread通过向量化加载其中的vec_size=4行。

3. 与`WN_per_loop`的关键区别

虽然两者在概念上相似，但有**关键区别**：

1. **内存布局不同**：
   - A矩阵在共享内存中是**转置的**（BK×BM布局）
   - B矩阵在共享内存中是**正常的**（BK×BN布局）

2. **线程索引计算不同**：
   ```cpp
   // A矩阵（M维度） - 转置访问
   tm_vec_id = lane_id >> get_log2x<WN_per_loop/vec_size>();  // 右移位

   // B矩阵（N维度） - 正常访问
   tn_vec_id = lane_id & (WN_per_loop/vec_size-1);  // 掩码操作
   ```

4. 详细图示说明

4.1 全局视图（Block级别）
```
BM = 256行

┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│                                                                                                                │
│  WM=64 行                                                                                                      │
│  ┌──────────────────────────────┐                                                                              │
│  │                              │  ← Warp 0 (wm_id=0, wn_id=0)                                                 │
│  │                              │                                                                              │
│  └──────────────────────────────┘                                                                              │
│  WM=64 行                                                                                                      │
│  ┌──────────────────────────────┐                                                                              │
│  │                              │  ← Warp 1 (wm_id=1, wn_id=0)                                                 │
│  │                              │                                                                              │
│  └──────────────────────────────┘                                                                              │
│  WM=64 行                                                                                                      │
│  ┌──────────────────────────────┐                                                                              │
```

```
  │  │              │  ← Warp 2 (wm_id=2, wn_id=0)                  │
  │  │              │                                  │
  │  └──────────────────────────────┘                                  │
  │   WM=64 行                                          │
  │  ┌──────────────────────────────┐                                  │
  │  │              │  ← Warp 3 (wm_id=3, wn_id=0)                  │
  │  │              │                                  │
  │  └──────────────────────────────┘                                  │
  │                                                     │
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
```

4.2 Warp级别（WM=64行）

```
一个Warp的WM=64行如何分配给32个Thread：
  WM_per_loop = 32行/循环

  循环0 (tm_loop=0):                循环1 (tm_loop=1):
  ┌──────────────────────────────────────┐
┌──────────────────────────────────────┐
  │   行0-31          |             |   行32-63         |
  |   32个Thread       |             |   32个Thread       |
  |   每个Thread加载    |             |   每个Thread加载    |
  |   vec_size=4行      |             |   vec_size=4行      |
  └──────────────────────────────────────┘
└──────────────────────────────────────┘
```

4.3 线程分配模式

由于A矩阵是**转置的**，线程分配与B矩阵不同：

```cpp
// tm_vec_id的计算
tm_vec_id = lane_id >> get_log2x<WN_per_loop/vec_size>();
// WN_per_loop/vec_size = 16/4 = 4, log2(4)=2
// 所以：tm_vec_id = lane_id >> 2
```

这意味着：
- `tm_vec_id` 取 `lane_id` 的高3位（因为右移2位）
- 32个线程被分成 **8组**（32/4=8），每组4个线程
- 每组的4个线程有相同的 `tm_vec_id`

```
```

lane_id分布：
lane_id: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
右移2位: 0 0 0 0 1 1 1 1 2 2 2 2 3 3 3 3 4 4 4 4 5 5 5 5 6 6 6 6 7 7 7 7
tm_vec_id:0 0 0 0 1 1 1 1 2 2 2 2 3 3 3 3 4 4 4 4 5 5 5 5 6 6 6 6 7 7 7 7
```

### 4.4 具体加载的行数据

当 `tm_loop=0`（加载前32行）时：

```
WM_per_loop=32行分为8个组（因为tm_vec_id=0..7）：
组0 (tm_vec_id=0): 行0-3     ← 线程0,1,2,3
组1 (tm_vec_id=1): 行4-7     ← 线程4,5,6,7
组2 (tm_vec_id=2): 行8-11    ← 线程8,9,10,11
组3 (tm_vec_id=3): 行12-15   ← 线程12,13,14,15
组4 (tm_vec_id=4): 行16-19   ← 线程16,17,18,19
组5 (tm_vec_id=5): 行20-23   ← 线程20,21,22,23
组6 (tm_vec_id=6): 行24-27   ← 线程24,25,26,27
组7 (tm_vec_id=7): 行28-31   ← 线程28,29,30,31
```

4.5 可视化加载图

```
WM = 64行，分为2次循环加载（tm_loop=0,1）：

循环0 (tm_loop=0): 行0-31
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│  组0    组1    组2    组3    组4    组5    组6    组7     │
│ (0-3)  (4-7) (8-11)(12-15)(16-19)(20-23)(24-27)(28-31)    │
│  t0-3  t4-7  t8-11 t12-15 t16-19 t20-23 t24-27 t28-31     │
│   4线   4线    4线    4线    4线    4线    4线    4线      │
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘

循环1 (tm_loop=1): 行32-63
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│  组0    组1    组2    组3    组4    组5    组6    组7     │
│ (32-35)(36-39)(40-43)(44-47)(48-51)(52-55)(56-59)(60-63)  │
│  t0-3  t4-7  t8-11 t12-15 t16-19 t20-23 t24-27 t28-31     │
│   4线   4线    4线    4线    4线    4线    4线    4线      │
```
```

└──┘
```

## 5. 内存访问模式分析

### 5.1 A矩阵的特殊性（转置存储）

A矩阵在共享内存中是**转置存储**的：
```cpp
// 共享内存布局：sA[BK][BM] = sA[16][256]
// 加载代码：get_offset(vec_t, tAsA, kid, m, BM)[0]

// 关键：tAsA的布局是 [WK][WM] = [8][64]
// 但是注意：BM是列主序的ld（leading dimension）
```

这意味着：
- 当线程访问 `tAsA[kid][m]` 时
- 实际上是在访问：`sA[wk_id*WK + kid][wm_id*WM + m]`
- 由于A是转置的，所以 `kid` 对应K维度，`m` 对应M维度

5.2 计算示例

代码中的关键计算：
```cpp
int m = tm_loop * WM_per_loop + tm_vec_id * vec_size;
int _m = tm_loop * vec_size;
```

示例1：线程0-3 (tm_vec_id=0)
```
tm_loop=0: m = 0*32 + 0*4 = 0    → 加载行0-3
tm_loop=1: m = 1*32 + 0*4 = 32   → 加载行32-35
```

示例2：线程4-7 (tm_vec_id=1)
```
tm_loop=0: m = 0*32 + 1*4 = 4    → 加载行4-7
tm_loop=1: m = 1*32 + 1*4 = 36   → 加载行36-39
```

示例3：线程28-31 (tm_vec_id=7)
```
tm_loop=0: m = 0*32 + 7*4 = 28   → 加载行28-31
tm_loop=1: m = 1*32 + 7*4 = 60   → 加载行60-63
```
```

```

### 5.3 寄存器布局对应

```
寄存器tArA的布局：WK×TM = 8×8
每个线程有8个寄存器元素（TM=8），通过2次循环加载：

循环0 (tm_loop=0): 加载到寄存器位置0-3
循环1 (tm_loop=1): 加载到寄存器位置4-7

寄存器索引：_m = tm_loop * vec_size
所以：
tm_loop=0: _m=0 → 寄存器位置0-3
tm_loop=1: _m=4 → 寄存器位置4-7
```

## 6. 为什么A矩阵需要不同的划分?

### 6.1 转置带来的内存访问特性

```
共享内存中A的布局：sA[BK][BM] = sA[16][256]
这是列主序存储!

当固定kid（K维度）时，不同m（M维度）的访问：
sA[kid][0], sA[kid][1], sA[kid][2], ... sA[kid][255]

这些元素在内存中是连续的!
因此，让tm_vec_id相同的线程访问相邻的行（m相差1）
可以实现合并访问。
```

6.2 合并访问优化

```
理想的合并访问模式：
warp中的32个线程访问连续32个内存地址

对于tm_vec_id=0的4个线程（线程0,1,2,3）：
线程0: sA[kid][0:4]  ← 行0-3
线程1: sA[kid][4:8]  ← 行4-7
线程2: sA[kid][8:12]  ← 行8-11
线程3: sA[kid][12:16] ← 行12-15
```
```

这4个线程的访问是连续的：行0-15
虽然不是完美连续的32个地址，但已经是较好的合并。
```

### 6.3 与B矩阵访问模式的对比

```
B矩阵（正常布局）：
内存连续的是列维度（N）
所以让tn_vec_id相同的线程访问连续的列

A矩阵（转置布局）：
内存连续的是行维度（M）
所以让tm_vec_id相同的线程访问连续的行

这就是为什么两者线程分配方式不同的根本原因！
```

7. 完整的物理意义总结

`WM_per_loop = 32` 的完整物理意义：

1. **循环粒度**：每个循环处理WM_per_loop=32行数据
2. **线程协作**：32个线程协作加载这32行数据
3. **向量化加载**：每个线程一次加载vec_size=4行
4. **内存优化**：针对转置的A矩阵布局，优化合并访问
5. **寄存器管理**：将TM=8行数据分2次加载到寄存器

8. 与WN_per_loop的对称性总结

特性	WM_per_loop (A矩阵)	WN_per_loop (B矩阵)
值	32	16
总数据量	WM=64行	WN=32列
循环次数	WM/WM_per_loop=2	WN/WN_per_loop=2
每组线程数	4线程/组	8线程/组
组数	8组	4组
线程索引	tm_vec_id = lane_id>>2	tn_vec_id = lane_id&3
内存布局	转置(BK×BM)	正常(BK×BN)
连续维度	行维度(M)连续	列维度(N)连续

这种设计确保了：
- **A矩阵**：利用转置布局的行连续性，优化内存访问
```

- **B矩阵**：利用正常布局的列连续性，优化内存访问
- **总体**：最大化内存带宽利用率，为高性能矩阵乘法奠定基础

//————————————

//—-—-—-

//—-—-—-